

MANUAL DEL DESARROLLADOR

Desarrollo de Aplicaciones Java Web con MVC & Jakarta EE 11

Institución: I.E.S.T.P. "Andrés Avelino Cáceres Dorregaray" (Huancayo)

Área Académica: Programa de Estudios de Diseño y Programación Web

Unidad Didáctica: Lenguaje de Programación | **Docente:** Mg. Ing. Raúl Fernández Bejarano

1. Arquitectura de Aplicaciones Java Web

En el desarrollo moderno, una **aplicación web** se define como un conjunto de recursos dinámicos y estáticos que se ejecutan sobre un servidor de aplicaciones y son accesibles mediante el protocolo HTTP desde un cliente (navegador web).

Ecosistema Jakarta EE: Anteriormente conocido como Java EE, representa el estándar empresarial para la construcción de software robusto y escalable en el lado del servidor, utilizando componentes nativos como Servlets y JSPs.

1.1 El Modelo Cliente-Servidor

La arquitectura fundamental se basa en un ciclo continuo de peticiones y respuestas:

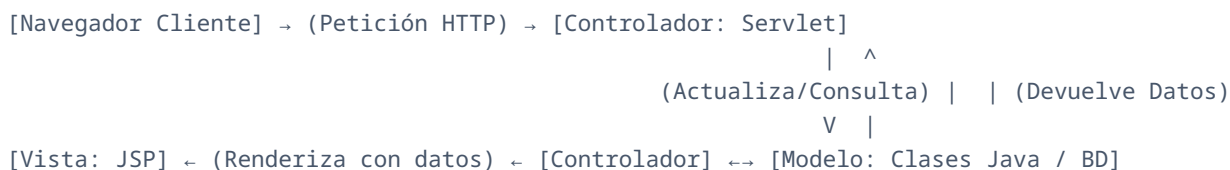
- **Cliente (Navegador):** Realiza peticiones de recursos (HTTP Request) como documentos HTML, imágenes o datos dinámicos.
- **Servidor de Aplicaciones (e.g., Apache Tomcat):** Procesa la petición, ejecuta código Java en el servidor para generar una respuesta dinámica, y la devuelve encapsulada como una respuesta HTTP (HTTP Response).

Característica	Páginas Estáticas (HTML / CSS / JS)	Páginas Dinámicas (JSP / Servlets)
Procesamiento	Se entregan directamente del disco del servidor al cliente sin cambios.	Se procesan y compilan en tiempo real en la Máquina Virtual de Java (JVM).
Interactividad	Limitada a lo programado en el lado del cliente.	Alta, conectada a bases de datos y APIs en tiempo real.
Rendimiento	Inmediato, requiere muy bajo uso de procesamiento en el servidor.	Requiere un contenedor de páginas web (Web Container) para su ejecución.

2. Patrón de Arquitectura Modelo-Vista-Controlador (MVC)

El patrón MVC es el estándar de oro en el desarrollo Java Web para garantizar el desacoplamiento y la mantenibilidad del código fuente. Divide la aplicación en tres componentes bien definidos:

Flujo de Arquitectura MVC en Java Web



- **Modelo (Model):** Representa los datos y la lógica de negocio de la aplicación. En Java, se implementa mediante clases estándar (POJO), clases de entidad y repositorios que acceden a la base de datos.
- **Vista (View):** Es la interfaz de usuario con la que interactúa el operador. Se implementa principalmente con archivos **JavaServer Pages (JSP)**, compuestos por HTML enriquecido con expresiones dinámicas.
- **Controlador (Controller):** Actúa como intermediario. Recibe las peticiones HTTP mediante **Java Servlets**, invoca la lógica en el Modelo y redirige al usuario hacia la Vista adecuada con los datos procesados.

3. Preparación del Entorno (Apache Tomcat)

Para ejecutar aplicaciones Java Web, requerimos un contenedor de servlets. En este manual utilizaremos **Apache Tomcat 11** (alineado con Jakarta EE 11).

1. **Descarga:** Obtenga el archivo comprimido (.zip o .tar.gz) desde el sitio web oficial de Apache Tomcat.
2. **Descompresión:** Extraiga el contenido en un directorio local seguro (por ejemplo: C:\tomcat11 o en su carpeta de usuario).
3. **Integración con el IDE (NetBeans):**
 - Abra Apache NetBeans.
 - Vaya a la pestaña de **Services** (generalmente al lado de Projects).
 - Haga clic derecho en **Servers** y seleccione **Add Server**.
 - Seleccione **Apache Tomcat or TomEE**.
 - Indique la ruta donde descomprimió el servidor y configure las credenciales de administrador de ser necesario.

4. Guía Práctica: Creación del Proyecto con Maven

Siga estos pasos estructurados para inicializar de forma correcta un proyecto Java Web utilizando el gestor de proyectos Maven en Apache NetBeans:

Paso 1: Inicialización del Asistente

Diríjase al menú superior y seleccione **File → New Project...** En el cuadro de diálogo, seleccione la categoría **Java with Maven** y luego el tipo de proyecto **Web Application**. Presione el botón **Next**.

Paso 2: Coordenadas del Proyecto

Configure los identificadores del proyecto según el estándar Maven:

- **Project Name:** CEjemplo09-1.0-SNAPSHOT (o el nombre descriptivo de su práctica).

- **Group Id:** Define el espacio de nombres de su organización (ej. `pe.edu.institutocajas`).
- **Package:** Paquete raíz por defecto para sus clases de código fuente (ej. `pe.edu.institutocajas.cejemplo09`).

Paso 3: Asociación del Servidor y Configuración de Jakarta EE

En la ventana de configuración del servidor:

- **Server:** Seleccione su servidor local previamente configurado (ej. Apache Tomcat).
- **Java EE Version:** Asegúrese de utilizar una especificación compatible como **Jakarta EE 11 Web** o la recomendada para su versión de Tomcat instalada.

¡Atención! Si el servidor no se encuentra asignado de forma correcta en este paso, NetBeans no cargará las dependencias de Jakarta Web, provocando que los archivos JSP o los Servlets muestren errores de compilación inmediatos.

Paso 4: Estructura Estándar del Proyecto Generado

Al finalizar el asistente, se creará la estructura de directorios del proyecto web:

```
Nombre-Del-Proyecto/ ├── Web Pages/ <-- Archivos Web (JSP, HTML, CSS, JS) | ├── META-INF/ | └── WEB-INF/ <-- Configuraciones protegidas (web.xml) ├── Source Packages/ <-- Código Java (Clases, Servlets, Controladores) ├── Other Sources/ ├── Dependencies/ <-- Librerías gestionadas por Maven (POM.xml) └── Project Files/ <-- Archivo pom.xml de configuración
```

Paso 5: Creación del Archivo de Vista (index.jsp)

Para garantizar la ejecución dinámica, sustituiremos la página HTML estática predeterminada:

1. Haga clic derecho sobre la carpeta **Web Pages** → **New** → **Other...**
2. Seleccione la categoría **Web** y el tipo de archivo **JSP**. Haga clic en **Next**.
3. Nombre al archivo como `index` y haga clic en **Finish**.
4. **Importante:** Elimine el archivo estático redundante `index.html` de la carpeta Web Pages para que el servidor sirva por defecto su nueva página dinámica `index.jsp`.

Paso 6: Despliegue y Ejecución

Haga clic derecho sobre el nodo principal del proyecto y seleccione **Run**. NetBeans compilará el código fuente, generará el empaquetado `.war`, levantará los servicios de Apache Tomcat y abrirá automáticamente el navegador de internet mostrando su aplicación web en ejecución activa.